

ClassyFM Public JSON API

The public, read-oriented JSON API for the ClassyFM mobile app. All endpoints are mounted under `/api/v1`.

- **Base URL:** `https://classyfm.co.id/api/v1`
 - **Format:** JSON (`Content-Type: application/json; charset=utf-8`)
 - **Orientation:** read-only, **cookie-free** — the only writes are the Connect chat endpoints, which authenticate with a Bearer token rather than a session cookie.
-

Table of Contents

- Conventions
 - Response envelopes
 - Errors
 - Caching
 - Pagination & filtering
 - URLs & images
 - Degraded mode
 - Index of Endpoints
 - App bootstrap endpoints
 - Content endpoints
 - Live endpoints
 - Playing the live stream in an app
 - When the backend API is unreachable
 - Advertising endpoints
 - Connect chat endpoints
 - Signing in from a mobile app
 - Authentication
 - Object reference
-

Conventions

Response envelopes

Shape	Used by	Body
List	roster/list endpoints	<code>{ "data": [...] }</code>
Paginated	<code>GET /news?source=...</code> , <code>GET /podcasts</code>	<code>{ "data": [...], "meta": { "page": 1, "total_pages": 3, "total": 27 } }</code>
Object	detail & aggregate endpoints	the object directly (no wrapper)

Errors

Errors return `{ "error": "message" }` with the relevant HTTP status:

Status	Meaning
400	Bad or empty request body, invalid path id
401	Authentication required / invalid Google or Bearer token
403	Banned from chat
404	Resource not found, or the feature is not configured
429	Rate limit exceeded
500	Server / database error
503	Chat temporarily unavailable

The two rate-limited endpoints (`POST /api/v1/connect/session` at 30/min and `POST /api/v1/connect/messages` at 20/min) are limited **per client IP** in a **fixed 60-second window**. Over-limit requests return `429` with a `Retry-After: 60` header and the JSON body `{ "error": "too many requests, try again later" }`.

Caching

Each response carries a `Cache-Control` header:

- `public, max-age=60` — slowly-changing content (programs, news, podcasts, about, ads, home, config).
- `no-store` — live or per-user data (now-playing, schedule state, TikTok live, all chat endpoints).

Pagination & filtering

Pagination applies only to `GET /news?source=...` and `GET /podcasts`:

- `page` — 1-based, defaults to `1` (values `< 1` are treated as `1`).
- Page size is fixed at `12`.
- `meta.total_pages` is at least `1` even when empty.

URLs & images

All image and link fields are returned as **absolute URLs** (resolved against the site origin, `https://classyfm.co.id`). Already-absolute URLs — e.g. aggregated news thumbnails — pass through unchanged.

Degraded mode

If the database is unavailable, **list** endpoints degrade gracefully to `{ "data": [] }` and **detail** endpoints return `404` — they never 500 for a missing DB.

Endpoints

App bootstrap endpoints

The two aggregate endpoints an app hits first at launch.

Method	Endpoint	Auth	Description
GET	/api/v1/config	open	App bootstrap: identity, stream, social, chat flag
GET	/api/v1/home	open	Aggregate home-screen feed in one request

Content endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/programs	open	Active program roster, each flagged on-air
GET	/api/v1/programs/{slug}	open	One program with weekly schedule & broadcasters
GET	/api/v1/broadcasters	open	Active broadcaster roster
GET	/api/v1/broadcasters/{slug}	open	One broadcaster plus the programs they present
GET	/api/v1/news	open	Grouped news preview, or one source paginated
GET	/api/v1/news/{slug}	open	One <code>hot_release</code> article with gallery & related
GET	/api/v1/podcasts	open	Published podcasts, paginated
GET	/api/v1/podcasts/{slug}	open	One podcast with series & broadcasters
GET	/api/v1/podcast-series	open	Podcast series list
GET	/api/v1/about	open	About-page banner, segments & broadcaster preview

Live endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/now-playing	open	Stream now-playing metadata & on-air program
GET	/api/v1/schedule/today	open	Today's schedule with live on-air/progress state
GET	/api/v1/schedule/current	open	The currently on-air program
GET	/api/v1/tiktok/live	open	TikTok live status

Advertising endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/ads	open	Ad banners for a page, by placement slot

Connect chat endpoints

Method	Endpoint	Auth	Description
GET	/api/v1/connect/messages	open	Poll chat messages
POST	/api/v1/connect/session	open 30/min	Exchange a Google ID token for a Connect token
GET	/api/v1/connect/me	Bearer	The chat identity behind the token
POST	/api/v1/connect/messages	Bearer 20/min	Post a chat message

App bootstrap endpoints

Both are `GET`, cached `public, max-age=60` — the endpoints an app calls at launch.

`GET /api/v1/config`

App bootstrap: station identity, stream URL, social links, and whether chat sign-in is available.

```
{
  "station": { "name": "Classy 103.4 FM", "slogan": "" },
  "stream_url": "https://c4.siar.us:10340/stream.mp3",
  "social": { "instagram": "https://...", "youtube": "https://..." },
  "connect": { "enabled": true }
}
```

`connect.enabled` is `true` only when Google sign-in is configured. `social` keys are platform names as configured in the admin panel.

`GET /api/v1/home`

Aggregate home-screen feed in a single request.

```
{
  "hero":    [ /* heroSlide objects */ ],
  "on_air":  { /* scheduleRow, or null */ },
  "programs": [ /* program objects, up to 6 */ ],
  "news":    [ /* newsGroup objects, up to 4 items each */ ]
}
```

See `heroSlide`.

Content endpoints

All are `GET` and cached `public, max-age=60`.

`GET /api/v1/programs`

Active program roster, each flagged with whether it is on air now.

```
{ "data": [  
  {  
    "title": "Morning Show",  
    "slug": "morning-show",  
    "description": "...",  
    "image_url": "https://classyfm.co.id/uploads/abc.jpg",  
    "url": "https://classyfm.co.id/program/morning-show",  
    "on_air": true  
  }  
] }
```

See `program` for the object fields.

`GET /api/v1/programs/{slug}`

One program with its full weekly schedule and broadcasters. `404` if the slug is unknown or the program is inactive.

```
{  
  "title": "Morning Show",  
  "slug": "morning-show",  
  "description": "...",  
  "image_url": "https://classyfm.co.id/uploads/abc.jpg",  
  "url": "https://classyfm.co.id/program/morning-show",  
  "on_air": true,  
  "schedule": [  
    { "from_day": 1, "to_day": 5, "start_time": "06:00", "end_time": "09:00", "host": "Anda, Yeni"  
  },  
  ],  
  "broadcasters": [ /* broadcaster objects */ ]  
}
```

`schedule[]` is a `scheduleGroup`; `broadcasters[]` are `broadcaster` objects.

GET /api/v1/broadcasters

Active broadcaster roster. `on_air` is `true` if any program the broadcaster presents is airing now.

```
{ "data": [ /* broadcaster objects */ ] }
```

GET /api/v1/broadcasters/{slug}

One broadcaster plus the programs they present. `404` if unknown.

```
{
  "name": "Anda",
  "slug": "anda",
  "role": "Announcer",
  "photo_url": "https://classyfm.co.id/uploads/anda.jpg",
  "bio": "...",
  "birth_place": "Padang",
  "birth_date": "...",
  "instagram": "...",
  "twitter": "...",
  "facebook": "...",
  "url": "https://classyfm.co.id/broadcasters/anda",
  "on_air": false,
  "programs": [ /* program objects */ ]
}
```

GET /api/v1/news

Two modes:

Without `source` (or with an unrecognized one) — grouped preview, up to 6 items per source group:

```
{ "data": [
  { "source": "klikpositif", "label": "KlikPositif", "items": [ /* news items */ ] },
  { "source": "katasumbar", "label": "KataSumbar", "items": [ ... ] },
  { "source": "hot_release", "label": "Hot Release", "items": [ ... ] },
  { "source": "youtube", "label": "YouTube", "items": [ ... ] }
] }
```

With a valid `source` — that source's items, paginated:

Query param	Notes
<code>source</code>	One of <code>youtube</code> , <code>klikpositif</code> , <code>katasumbar</code> , <code>hot_release</code> . Invalid values fall back to grouped mode.
<code>page</code>	1-based, page size 12.

```
{ "data": [ /* news items */ ], "meta": { "page": 1, "total_pages": 4, "total": 42 } }
```

Items are `newsItem` objects. Note: `hot_release` items link to the on-site `/news/{slug}`; other (aggregated) sources keep their external `url`.

GET `/api/v1/news/{slug}`

One `hot_release` article, with split paragraphs, a mid-article image gallery, and related articles. `404` if unknown (only `hot_release` articles have on-site detail).

```
{
  "source": "hot_release",
  "source_label": "Hot Release",
  "title": "...",
  "slug": "...",
  "excerpt": "...",
  "image_url": "https://...",
  "url": "https://classyfm.co.id/news/...",
  "published_at": "2026-08-09T10:00:00Z",
  "is_featured": true,
  "paragraphs": ["...", "..."],
  "gallery_index": 3,
  "middle_images": ["https://...", "https://..."],
  "related": [ /* news items */ ]
}
```

`gallery_index` is the paragraph index after which the `middle_images` gallery should be inserted.

GET `/api/v1/podcasts`

Published podcasts, newest first, paginated.

Query param	Notes
<code>series</code>	Optional series slug filter. Invalid slugs are ignored (returns all).
<code>page</code>	1-based, page size 12.

```
{ "data": [ /* podcast objects */ ], "meta": { "page": 1, "total_pages": 2, "total": 18 } }
```

GET /api/v1/podcasts/{slug}

One podcast with its series name and broadcasters. 404 if unknown.

```
{
  "title": "...",
  "slug": "...",
  "description": "...",
  "spotify_url": "https://open.spotify.com/...",
  "thumb_url": "https://...",
  "series_name": "...",
  "url": "https://classyfm.co.id/podcast/...",
  "broadcasters": [ /* broadcaster objects */ ]
}
```

GET /api/v1/podcast-series

Podcast series list (for a filter picker feeding ?series=).

```
{ "data": [ { "name": "...", "slug": "..." } ] }
```

GET /api/v1/about

About-page content: banner, text segments, and a broadcaster preview (max 8).

```
{
  "banner": {
    "media_type": "video",
    "image_url": "https://...",
    "video_url": "https://youtu.be/...",
    "embed_url": "https://www.youtube.com/embed/..."
  },
  "segments": [ { "segment": "profile", "title": "...", "body": "..." } ],
  "broadcasters": [ /* broadcaster objects */ ]
}
```

`banner.embed_url` is present only when `media_type` is `"video"` and the video URL is a resolvable YouTube link. `segment` is one of `profile`, `music`, `audience`.

Live endpoints

All are `GET`, cached `no-store` — live stream state and schedule progress.

`GET` `/api/v1/now-playing`

Stream now-playing metadata, plus the on-air program when the stream is live.

```
{
  "artist": "...",
  "song": "...",
  "has_song": true,
  "cover_url": "https://...",
  "live": true,
  "program": { /* scheduleRow, present only when live and a program is on air */ }
}
```

See `scheduleRow`. (The Shoutcast listener count is deliberately not exposed here — it is admin-only.)

`GET` `/api/v1/schedule/today`

Today's full schedule with live on-air/progress state.

```
{ "data": [ /* scheduleRow objects */ ] }
```

`GET` `/api/v1/schedule/current`

The currently on-air program as a single `scheduleRow` object, or `{ "on_air": false }` when nothing is airing.

`GET` `/api/v1/tiktok/live`

```
{ "live": true, "title": "..." }
```

Playing the live stream in an app

The audio stream is a **direct external Shoutcast MP3** — it lives on the Shoutcast host, **not** on `classyfm.co.id`, and this API never proxies or redirects the audio. The app plays it **directly**:

1. **Bootstrap the URL.** Read `stream_url` from `/api/v1/config` once at startup and hand it to the device's native audio player. It is a plain, unauthenticated streaming MP3 (Shoutcast) — no headers,

no token, no proxy. Prefer reading it from `config` over hardcoding it, so the Shoutcast host can be moved server-side without an app release.

2. Drive now-playing from a poll loop — via this API, never Shoutcast directly.

`/api/v1/now-playing` on a timer to update the now-playing UI (and any lock-screen / notification metadata). The API fetches and caches the track metadata and `live` state from the Shoutcast server for you, so the app avoids CORS and doesn't hammer the Shoutcast box — **do not scrape the Shoutcast endpoints yourself**. The response is `no-store` but the server refreshes its upstream metadata only every ~12 s, so **polling faster than ~15 s gains nothing** — settle on roughly a 15 s interval while the player is active, and pause polling when it is stopped or backgrounded without audio.

- Show `artist` + `song` when `has_song` is `true`; when `false` there is no track metadata (station ID / no title) — fall back to the station name.
- Use `cover_url` for artwork, but it is best-effort and may be `""` — fall back to a bundled placeholder or the on-air program image.

3. Handle live vs. off-air.

Switch the UI between "on air" and "off air" on the `live` flag. When `live` is `true`, the optional `program` object (a `scheduleRow`) gives the current show, host, and `progress` (0–100); `/api/v1/schedule/current` returns the same thing standalone, and `/api/v1/schedule/today` backs an "up next" list.

When the backend API is unreachable

Because the audio is a **direct Shoutcast MP3** and this API never proxies it, backend downtime (network error, timeout, `5xx`) does **not** interrupt playback — only the *metadata* around it. Keep the audio running and degrade only the now-playing chrome:

1. **Persist `config`**. Cache the last successful `/api/v1/config` response (at minimum `stream_url`, plus station name/slogan and social links) in local storage. At launch, start playback from the cached `stream_url` even when `config` can't be re-fetched. There is **no bundled/hardcoded stream URL** — the app never invents one. On a true cold start (first-ever launch, nothing cached, and the API unreachable) there is no URL to play: show a "stream unavailable / retry" state and fetch `config` again once connectivity returns.
2. **Keep audio alive when `now-playing` fails**. A failed/timed-out/`5xx` `/api/v1/now-playing` poll is a metadata gap, not a stream failure — never stop or reset the player because of it. Hold the last-known `artist` / `song` and `live` state, or fall back to the station name + bundled placeholder art.
3. **Back off, then recover**. On repeated poll failures, widen the interval (e.g. exponential backoff up to ~60 s) instead of hammering. On the next successful poll, resume the normal ~15 s cadence, refresh the now-playing UI, and re-cache `config`. No user action or app restart should be required.

A live listener count is intentionally not available to apps.

Advertising endpoints

GET, cached `public, max-age=60`.

GET `/api/v1/ads`

Ad banners for a page, grouped into `top` and `bottom` placement slots.

Query param	Notes
<code>page</code>	Target page key. Invalid keys return <code>400</code> . Omitted returns only banners targeted at every page.

Valid `page` keys: `home`, `about`, `program`, `program_detail`, `live`, `news`, `news_detail`, `broadcasters`, `broadcaster_detail`.

```
{
  "top": {
    "slideshow": false,
    "rotate_ms": 6000,
    "placeholder": false,
    "placeholder_text": "",
    "banners": [
      {
        "image_url": "https://classyfm.co.id/uploads/ad.jpg",
        "link_url": "https://sponsor.example",
        "alt": "...",
        "title": "..."
      }
    ]
  },
  "bottom": { /* same shape */ }
```

Each slot's `banners` is an array (empty when the slot has none). `slideshow` tells the client to rotate banners every `rotate_ms` rather than stack them; `placeholder` (with `placeholder_text`) says an empty slot should hold its space rather than collapse.

Connect chat endpoints

The Connect chatroom over JSON, mounted under `/api/v1/connect`. **Reads are open; writes require a Bearer token** obtained by exchanging a Google ID token (see [Authentication](#)). All chat responses are `no-store`.

GET `/api/v1/connect/messages` **OPEN**

Poll chat messages.

Query param	Notes
<code>since</code>	Optional message id. With it, returns up to 200 messages newer than that id (delta poll). Without it, returns the 50 most recent.

```
{ "messages": [ /* chatMessage objects */ ] }
```

POST `/api/v1/connect/session` **OPEN** **RATE-LIMITED 30/MIN**

Exchange a native Google ID token for a Connect session token. The app obtains the ID token via the platform's own Google sign-in (so Google OAuth never runs in a WebView), passing the website's Google client id as its `serverClientId`.

Request:

```
{ "id_token": "<google-id-token>" }
```

Response:

```
{
  "token": "<connect-session-token>",
  "user": { "id": 42, "name": "...", "avatar": "https://...", "is_admin": false }
}
```

The `token` is verified server-side against Google's `tokeninfo` endpoint (checking audience and issuer), then the chat user is upserted and the admin badge recomputed. Send `token` as `Authorization: Bearer <token>` on subsequent authenticated calls.

Errors: `404` if chat login is not configured, `400` if `id_token` is missing, `401 invalid Google token`, `500` on failure. The request body is capped at 16 KiB.

GET /api/v1/connect/me BEARER REQUIRED

Returns the chat identity behind the token.

```
{ "user": { "id": 42, "name": "...", "avatar": "https://...", "is_admin": false } }
```

POST /api/v1/connect/messages BEARER REQUIRED RATE-LIMITED 20/MIN

Post a chat message. The body is sanitized, trimmed, and capped at **1000 characters**.

Request:

```
{ "body": "Hello!" }
```

Response — the stored message, so the app can render it optimistically:

```
{ "message": { "id": 101, "user_id": 42, "name": "...", "avatar": "...", "is_admin": false, "body": "H
```

Errors: 401 if unauthenticated, 403 your account is blocked from chat if banned, 400 if the body is empty/invalid, 503 if chat is unavailable. Body capped at 16 KiB.

Signing in from a mobile app

The app-side steps. The token mechanics behind them — verification, TTL, storage, re-auth — live in [Authentication](#); this is just the order of operations:

1. **Check availability.** Only offer sign-in when `/api/v1/config` reports `connect.enabled: true`; when it is `false`, Google sign-in is not configured server-side and `/connect/session` returns 404. (Reads — polling messages — need no sign-in; require it only before posting.)
2. **Sign in on-device.** Run the platform's native Google sign-in (never a WebView) and receive a Google **ID token**, passing the ClassyFM Google client id as the `serverClientId` — see [Authentication](#) for that value and why.
3. **Exchange the ID token.** POST `{ "id_token": "..." }` here and store the returned Bearer `token` per [Authentication](#). The `user` object is enough to render the signed-in identity immediately.
4. **Use the token.** Send `Authorization: Bearer <token>` on `/connect/me` and POST `/connect/messages`.

Authentication

Chat writes use a self-contained, **HMAC-signed Bearer token** — no server-side session row, no cookie.

1. **Obtain a token.** The native app signs in with Google on-device and receives a Google **ID token**. It **POST**s that to `/api/v1/connect/session`, which verifies it against Google's `tokeninfo` endpoint (enforcing that the audience equals the site's configured Google client id and the issuer is Google), upserts the chat user, and returns a **Connect session token**. The app must pass this same Google client-id value as its `serverClientId` at sign-in — obtain it from the ClassyFM team.

2. **Use the token.** Send it on authenticated endpoints:

```
Authorization: Bearer <connect-session-token>
```

3. **Token properties.**

- Payload carries only the chat user id and an expiry, signed with `SESSION_SECRET`.
- **TTL: 30 days.**
- On each request the identity (name, avatar, admin/ban status) is re-read from the database, so bans and role changes take effect immediately without re-issuing the token.
- Enforcement is per-endpoint: `401` when the token is missing/invalid.

4. **App lifecycle.**

- **Storage & expiry.** Persist the token in the platform's secure store (not plain preferences). It is valid for 30 days; there is no refresh endpoint — re-run the Google sign-in exchange to get a fresh one.
- **Re-auth on `401`.** Any authenticated call may return `401` once the token expires or is otherwise invalid — clear the stored token and prompt sign-in again.
- **`403` is terminal.** `403 your account is blocked from chat` means the account is banned; do not retry or re-issue — the same identity will keep being rejected.

Object reference

Fields marked (*optional*) are omitted from the JSON when empty.

program

Field	Type	Notes
<code>title</code>	string	
<code>slug</code>	string	
<code>description</code>	string	(<i>optional</i>)
<code>image_url</code>	string	(<i>optional</i>) absolute URL
<code>url</code>	string	on-site program URL
<code>on_air</code>	bool	airing now

scheduleGroup

Field	Type	Notes
<code>from_day</code>	int	0 = Sunday ... 6 = Saturday
<code>to_day</code>	int	end of the day range (same numbering)
<code>start_time</code>	string	<code>HH:MM</code>
<code>end_time</code>	string	<code>HH:MM</code>
<code>host</code>	string	(<i>optional</i>) effective broadcaster set, e.g. <code>"Anda, Yeni"</code>

broadcaster

Field	Type	Notes
<code>name</code>	string	
<code>slug</code>	string	
<code>role</code>	string	<i>(optional)</i>
<code>photo_url</code>	string	<i>(optional)</i> absolute URL
<code>bio</code>	string	<i>(optional)</i>
<code>birth_place</code>	string	<i>(optional)</i>
<code>birth_date</code>	string	<i>(optional)</i>
<code>instagram</code>	string	<i>(optional)</i>
<code>twitter</code>	string	<i>(optional)</i>
<code>facebook</code>	string	<i>(optional)</i>
<code>url</code>	string	on-site broadcaster URL
<code>on_air</code>	bool	presenting an airing program now

newsItem

Field	Type	Notes
<code>source</code>	string	<code>youtube</code> <code>klikpositif</code> <code>katasumbar</code> <code>hot_release</code>
<code>source_label</code>	string	human-readable source name
<code>title</code>	string	
<code>slug</code>	string	<i>(optional)</i> present for <code>hot_release</code>
<code>excerpt</code>	string	<i>(optional)</i>
<code>image_url</code>	string	<i>(optional)</i>
<code>url</code>	string	on-site for <code>hot_release</code> , external otherwise
<code>published_at</code>	string	RFC 3339 timestamp
<code>is_featured</code>	bool	<i>(optional)</i>

newsGroup

Field	Type	Notes
<code>source</code>	string	source key
<code>label</code>	string	display label
<code>items</code>	newsItem[]	

podcast

Field	Type	Notes
<code>title</code>	string	
<code>slug</code>	string	
<code>description</code>	string	<i>(optional)</i>
<code>spotify_url</code>	string	<i>(optional)</i>
<code>thumb_url</code>	string	<i>(optional)</i> absolute URL
<code>series_name</code>	string	<i>(optional)</i>
<code>broadcaster</code>	string	<i>(optional)</i> list-view only
<code>url</code>	string	on-site podcast URL

scheduleRow

Field	Type	Notes
<code>title</code>	string	program title
<code>slug</code>	string	<i>(optional)</i>
<code>host</code>	string	<i>(optional)</i>
<code>image</code>	string	<i>(optional)</i> absolute URL
<code>start_time</code>	string	<code>HH:MM</code>
<code>end_time</code>	string	<code>HH:MM</code>
<code>on_air</code>	bool	
<code>progress</code>	int	0-100, % through the slot
<code>ended</code>	bool	slot already finished today

heroSlide

Field	Type	Notes
<code>href</code>	string	<i>(optional)</i> link target
<code>target_blank</code>	bool	<i>(optional)</i> open in new tab
<code>image_url</code>	string	<i>(optional)</i>
<code>badge_label</code>	string	<i>(optional)</i>
<code>title</code>	string	
<code>excerpt</code>	string	<i>(optional)</i>
<code>date</code>	string	<i>(optional)</i> RFC 3339 timestamp

connectUser

Field	Type	Notes
<code>id</code>	int	chat user id
<code>name</code>	string	
<code>avatar</code>	string	<i>(optional)</i>
<code>is_admin</code>	bool	

chatMessage

Field	Type	Notes
<code>id</code>	int	message id (use as <code>since</code> cursor)
<code>user_id</code>	int	author's chat user id
<code>name</code>	string	author name
<code>avatar</code>	string	author avatar URL
<code>is_admin</code>	bool	author is a moderator
<code>body</code>	string	sanitized message text
<code>time</code>	string	<code>HH:MM</code> , station timezone